Module 3 — Lab: SQL Analytics Pack

Build an analytical SQL query pack and KPI brief against a multi-table PostgreSQL database for Levant Tech Solutions, a fictional Jordanian technology company.

Due: Day 2 (Monday) end of day. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment:** Accept the M3 Lab assignment
2. Clone your repo and open it in VS Code.
3. Start your Postgres container (or reuse from EID 1):

```
docker run -d --name postgres-m3 \
  -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=levant_tech \
  -p 5432:5432 -v pgdata_m3:/var/lib/postgresql/data \
  postgres:15-alpine
```

4. Create the database and load the schema + seed data:

```
psql -h localhost -U postgres -d levant_tech -f schema.sql
psql -h localhost -U postgres -d levant_tech -f seed_data.sql
```

5. Verify: `psql -h localhost -U postgres -d levant_tech -c "\dt"` should show 4 tables: `departments`, `employees`, `projects`, `project_assignments`.
6. Install dependencies: `pip install -r requirements.txt`

The Database

Levant Tech Solutions has four tables:

Table	Description	Key columns
`departments`	Company departments	dept_id (PK), name, location, budget
`employees`	Staff members	emp_id (PK), name, dept_id (FK), hire_date, salary, title
`projects`	Company projects	project_id (PK), name, dept_id (FK), start_date, end_date, budget
`project_assignments`	Employee-project links	assignment_id (PK), emp_id (FK), project_id (FK), hours_allocated, role

Explore the data: run `SELECT COUNT(*) FROM employees;` and browse a few rows to understand the data before writing queries.

Part 1: SQL Queries

Write your queries in `queries.sql`. Each query has a numbered comment block — write your SQL below the corresponding marker.

Q1 — Employee Directory with Departments List all employees with their department name, sorted by department name (ascending) then salary (descending). Use a JOIN.

Q2 — Department Salary Analysis Calculate total salary expenditure by department. Only include departments with total salary exceeding 150,000. Use GROUP BY and HAVING.

Q3 — Highest-Paid Employee per Department For each department, find the employee with the highest salary. Use a window function (ROW_NUMBER or RANK) and filter to rank = 1.

Q4 — Project Staffing Overview List all projects with the number of assigned employees and total hours allocated. Include projects with zero assignments (they should show 0, not be excluded). Use LEFT JOIN and GROUP BY.

Q5 — Above-Average Departments Using a CTE, find departments where the average employee salary exceeds the company-wide average salary. Show department name, department average, and company average.

Q6 — Running Salary Total For each employee, show their salary and the running total of salaries within their department, ordered by hire date. Use a window function with SUM OVER.

Q7 — Unassigned Employees Find employees who are not assigned to any project. Use a LEFT JOIN with a NULL check (or NOT EXISTS).

Q8 — Hiring Trends Calculate month-over-month hire counts. Extract year and month from hire_date, count employees per period, and order chronologically. Use a CTE or subquery.

Q9 — Schema Design: Employee Certifications The company wants to track employee training certifications. Design and implement:

1. A `certifications` table with columns: `certification_id` (PK, SERIAL), `name` (VARCHAR, NOT NULL), `issuing_org` (VARCHAR), `level` (VARCHAR — e.g., 'Beginner', 'Intermediate', 'Advanced')
2. An `employee_certifications` bridge table with columns: `id` (PK, SERIAL), `emp_id` (FK → employees), `certification_id` (FK → certifications), `certification_date` (DATE, NOT NULL)

Write:

- The CREATE TABLE DDL for both tables with appropriate primary keys, foreign keys, and constraints
- INSERT statements adding at least 3 certifications and at least 5 employee-certification records (use existing emp_ids from the employees table)
- One query that lists all employees with their certifications, showing employee name, certification name, issuing organization, and certification date. Use a JOIN across employees, employee_certifications, and certifications.

Part 2: KPI Brief

In `kpi_brief.md`, define 3–5 Key Performance Indicators for Levant Tech's HR and project management. For each KPI:

- **Name:** A clear, specific KPI name
- **Definition:** How it's calculated (reference which query or tables)
- **Current value:** The actual number from your query results
- **Interpretation:** One sentence explaining what this number means for the business

Examples of good KPIs: department salary efficiency (salary per employee), project staffing ratio (employees per project), employee utilization rate (% assigned to projects).

Challenge Extensions

These are optional extensions for learners who complete the base requirements. Each tier introduces concepts or techniques not covered in the base work.

Tier 1 — Complex Analytics Queries

- Write a query identifying "at-risk" projects: projects where total allocated hours exceed 80% of the project budget (treat budget as available hours for simplicity). Requires a derived calculation and comparison across joined tables.
- Write a cross-department analysis query: find employees who are assigned to projects in departments other than their own. This requires a multi-table JOIN with a condition comparing `employees.dept_id` to `projects.dept_id`.

Tier 2 — Dynamic Reporting with Views and Functions

- Create PostgreSQL VIEWs for the most-used queries: a department summary view and a project status view. Explore the difference between standard and materialized views (`CREATE MATERIALIZED VIEW`).
- Write a PostgreSQL function (PL/pgSQL) that accepts a department name as input and returns a JSON object with: employee count, total salary, number of active projects. Call the function from psql and from Python.

Tier 3 — Schema Evolution and Migration

- Design a `salary_history` table to track salary changes over time. Define the schema, write the DDL, and seed it with realistic data (e.g., 2–3 salary records per employee spanning the last 3 years).
- Write a migration script that populates `salary_history` from the existing `employees` table (one initial record per employee at their current salary).
- Write queries against the new schema: salary growth rate by department over time, employees due for salary review (no change in 12+ months).
- Write a brief analysis: how would you handle this migration in production with live data? What are the risks of adding a new table and backfilling?

Submit

1. Create branch `lab-3/sql-analytics`
2. Commit `queries.sql` and `kpi_brief.md`
3. Open a PR to `main`
4. Your PR description must include:
   - Output of Q2 (department salary totals)
   - Output of Q3 (highest-salary employee per department)
   - Q9 CREATE TABLE DDL for the certifications schema
5. Paste your PR URL into TalentLMS → Module 3 → Lab 3 to submit this assignment.